TP 17 : Chercher les plus proches voisins.

L'entièreté du sujet est à réaliser en `OCaml`.

Préliminaires

Dans ce TP, on cherche à reconnaître des chiffres écrits à la main. Pour cela, il nous est fourni un ensemble d'images standardisées. Chaque image est composée de 28 pixels sur 28 pixels en noir et blanc. CHaque pixel est donc un niveau de gris, soit un entier entre 0 et 255. Le jeu de données est connu sous le nom de MNIST tiré du site suivant http://yann.lecun.com/exdb/mnist/. C'est un jeu de données assez classique pour tester des méthodes d'apprentissage supervisé. Le jeu d'entraînement contient 60000 images tandis que celui de test en contient 10000.

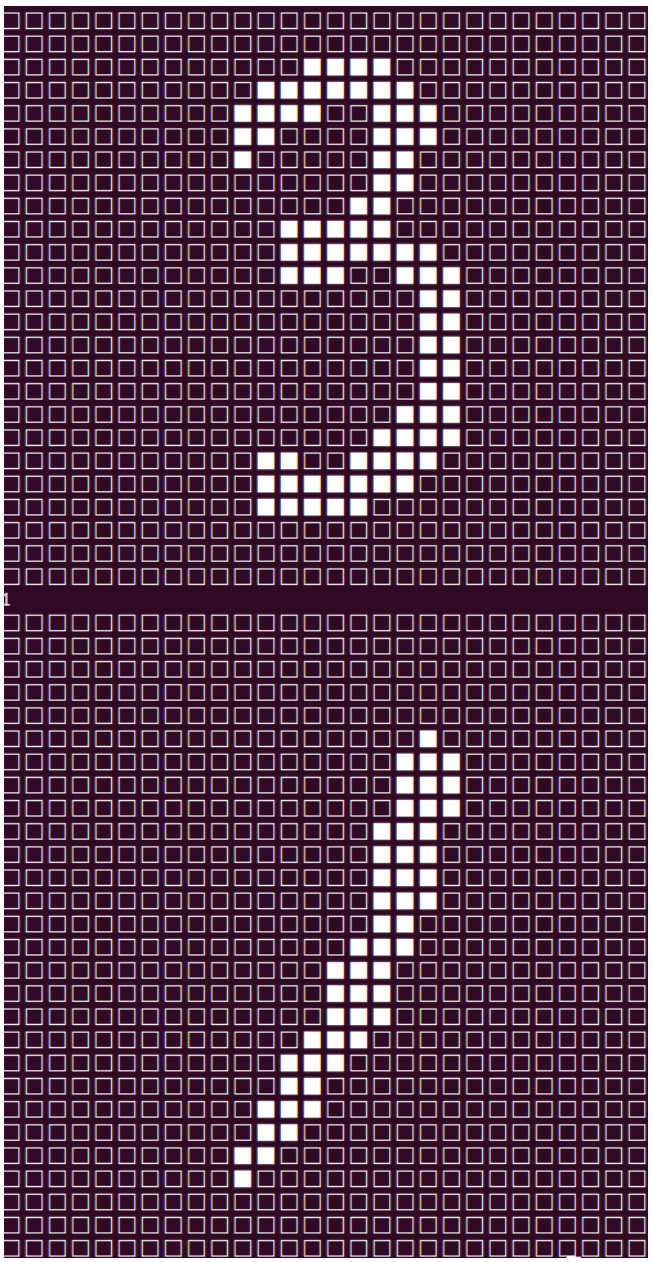

FIGURE 1 – Un affichage de deux images du jeu d'entraînement en représentant les valeurs inférieures à 123 en blanc et les autres en noir.

Les fichiers sont compressés, il faut tout d'abord les décompresser à l'aide de la commande `gzip -dk nom_fichier` puis en extraire les données à l'aide du fichier python `test.py`. Il est nécessaire d'exécuter la commande `pip install idx2numpy` avant d'utiliser celui-ci. Pour cela, il suffit de rendre exécutable le fichier `a_executer.sh` à l'aide de la commande `chmod +x a_executer.sh` et de l'exécuter à l'aide de `./a_executer.sh`.

On propose d'utiliser des arbres k-dimensionnels pour trouver les voisins les plus proches. Pour cela, on utilise les types suivants :

```
type image_etiquette = Etiquetage of (int array array) * int

type kdArbre = Vide | Noeud of image_etiquette * kdArbre * kdArbre
```

Ici, une image est une matrice d'entiers de taille 28 sur 28 et l'étiquette associée est un entier, celui qui correspond au chiffre écrit.

Construction

On cherche dans un premier temps à construire un arbre k-dimensionnel. Pour considérer les images comme un vecteur, on numérote la case (i, j) par la valeur $28i + j$.

Question 1 : Écrire une fonction `compter : image_etiquette array -> int -> int` qui, étant donné un tableau d'images étiquetées et un indice k, compte le nombre d'éléments dont la k-ième coordonnée est inférieure à celle du premier élément du tableau. On ne comptera pas celui-ci

Question 2 : Écrire une fonction `partitionner : image_etiquette array -> int -> (image_etiquette array * image_etiquette array)` qui, étant donné un tableau d'images étiquetées et une numéro de coordonnées, partitionne le tableau en deux. Pour cela, on utilisera le premier élément du tableau comme pivot et l'on comparera les éléments à l'aide de leur k-ème coordonnée. On renverra enfin deux tableaux, celui composé des éléments ayant une plus petite k-ème coordonnée que le pivot et celui composé des éléments en ayant une strictement plus grande. Le pivot ne sera présent dans aucun des deux tableaux.

Question 3 : Écrire une fonction `construire : image_etiquette array -> kdArbre` qui, étant donnés un tableau de n points de dimension d, construit un arbre k-dimensionnel. On utilisera toujours le premier élément comme pivot. On pourra utiliser une fonction auxiliaire qui recevra en paramètre supplémentaire la profondeur actuelle du nœud.

File de priorité

On se propose d'utiliser la distance de Manhattan pour estimer la proximité entre deux images.

Question 4 : Écrire une fonction `distance_Manhattan : int array array -> int array arrray -> int` qui, étant données deux images, calcule leur distance.
On cherche ici à construire une file de priorité. Puisqu'il ne s'agit pas ici du cœur du sujet, on choisit donc un méthode simple, celle de la liste triée par ordre décroissant selon la distance. Par soucis d'efficacité, on stocke dans cette liste des couples : l'élément concerné et la distance. Ce sera donc une `(image_etiquette * int)list`. On y ajoute un entier comptant le nombre d'éléments présents dans cette file de priorité et un autre comptant le nombre maximal d'éléments qu'elle peut accueillir. On utilisera finalement le type suivant :

```
type file_priorite = (image_etiquette * int) list * int * int
```

Question 5 : Écrire une fonction `distance_max : file_priorite -> int` qui renvoie la distance maximale d'un élément dans la file. On renverra `Int.max_int` dans le cas où la liste n'est pas remplie.

Question 6 : Écrire une fonction `inserer : file_priorite -> image_etiquette -> int -> file_priorite` qui étant donné un nouvel élément, l'insère dans la file de priorité. On veillera à tester si celle-ci est pleine et à comparer les distances des éléments déjà stockés avec le nouveau pour préserver les contraintes du type définies plus haut. L'entier fourni est la distance associée à l'élément inséré.

Recherche des plus proches voisins

On rappelle les grandes lignes de la recherche des plus proches voisins dans un arbre k-dimensionnel :

- On garde en mémoire une file de priorité contenant les plus proches voisins de x parmi les points étudiés, dans une limite de K au maximum.
- Chaque fois qu'on découvre un nouveau point a, on l'ajoute à la file de priorité si elle en contient moins que K, ou si ce nouveau point est plus proche de x que le voisin le plus loin. On extrait ce voisin le plus loin si nécessaire.
- On explore une partie de l'arbre d-dimensionnel selon l'idée suivante : pour chaque nœud a de l'arbre, on explore le fils gauche ou droit en premier selon où se trouve x par rapport à l'hyperplan de découpage en a. Après cette exploration, deux cas sont possibles :
  - soit on a déjà trouvé K voisins, et le plus loin d'entre eux est plus proche de x que ne l'est l'hyperplan de découpage, auquel cas on n'explore pas l'autre fils (car tous les points qui s'y trouvent seront trop loin) ;
  - sinon, on explore l'autre fils.

Question 7 : Écrire une fonction `distance_hyperplan : image_etiquette -> int array array -> int -> int` qui, étant donné une image x non étiquetée, calcule la distance de x à l'hyperplan passant par une image étiquetée y où seul la j-ème coordonnée est fixée.

Question 8 : Écrire une fonction `recherche_k_plus_proches : kdArbre -> int -> int array array -> ( image_etiquette * int)list` qui effectue la recherche des k plus proches voisins d'une image non étiquetée dans l'arbre k-dimensionnel. Cette fonction renverra la liste des k plus proches voisins.

Utilisation en pratique

Question 9 : Écrire une fonction `majoritaire : (image_etiquette * int)list -> int` qui, étant donnée une file de priorité obtenue par les fonctions précédentes, renvoie l'étiquette majoritaire.

Question 10 : Écrire une fonction `prediction : kdArbre -> int -> int array array` qui, étant donné un arbre k-dimensionnel et un entier k, renvoie les k plus proches voisins de l'image fournie.

On a fourni les fonctions suivantes :

- `recuperer_images_fichier: string -> int -> (int array array)list` qui, étant donné un nom de fichier et un entier k, renvoie la liste des k premières images du fichier sous forme de matrices.
- `etiqueter_liste : (int array array)list -> string -> int -> image_etiquette list` qui, étant donnés une liste d'images, un nom de fichier où trouver les étiquettes correspondantes et un nombre k, renvoie la liste des k premières images étiquetées.
- `prediction_reussie : int -> bool array` qui étant donnés un entier k, construit un arbre k-dimensionnel en lisant les fichiers fournis et l'utilise pour prédire les étiquettes sur le jeu de test. Elle renvoie alors une liste de booléens indiquant pour chaque image du jeu de test si elle a été correctement prédit.

Vous pouvez commencer par tester vos fonctions en vous restreignant à une portion des images du fichier de test afin de réduire le temps d'exécution.

Question 11 : Écrire une fonction `taux_erreur : int -> float` qui, étant donné un entier k, sur les fichiers fournis, estime la probabilité d'erreur de l'algorithme à l'aide du jeu de test.

Question 12 : Déterminer quelle valeur de k minimise le taux d'erreur.